
Diffusion Policies in Reinforcement Learning

Boyang Xia
2023533073

Jiawen Dai
2023533132

Zhichen Zhong
2023533131

Abstract

Offline reinforcement learning (RL) seeks to learn optimal policies from previously gathered static datasets, a paradigm critical for applications where online interaction is costly or risky. While Diffusion Q-Learning (Diffusion-QL) has shown state-of-the-art performance by using expressive diffusion models for policy representation, it faces computational inefficiencies and generality limitations. This paper investigates and addresses these challenges by exploring more efficient alternatives and their integration with online adaptation. We first review Efficient Diffusion Policy (EDP), which significantly accelerates diffusion policy training and broadens its compatibility with various offline RL algorithms through techniques like action approximation and DPM-Solver. We then introduce Diffusion Policy Policy Optimization (DPPO), a framework that enables robust online fine-tuning of pre-trained diffusion policies using policy gradient methods. Our experimental results include comprehensive reproductions of Diffusion-QL on toy examples, validating its capacity to handle multimodal action distributions. We extend these analyses to custom, complex toy bandit environments, where EDP also exhibits strong performance. Finally, we propose EDP-DPPO, a novel two-stage framework integrating EDP’s efficient offline pre-training with DPPO’s online fine-tuning.

1 Introduction

Offline Reinforcement Learning (Offline RL) presents a significant paradigm for learning decision-making policies from previously gathered datasets, with applications spanning autonomous driving, healthcare, and robotics. Our work is motivated by the potential of Diffusion Q-Learning (Diffusion-QL) to accurately restore the underlying distribution of offline datasets through its expressive policy representation. However, recognizing the computational intensity of this approach, we further investigate more efficient alternatives and explore their integration with subsequent online fine-tuning.

A fundamental challenge in offline RL is distributional shift: policies trained naively on static datasets may exploit errors in value function estimates for out-of-distribution (OOD) actions, leading to degraded performance during deployment. As demonstrated in the ICLR 2023 paper “Diffusion Policies as an Expressive Policy Class for Offline Reinforcement Learning” by Wang *et al.*, conventional policy regularization methods often fail because their policy classes (e.g., Gaussian policies) lack sufficient expressiveness to capture the complex, potentially multimodal behavior policies characteristic of offline datasets. This limitation frequently results in improper regularization and suboptimal solutions. Diffusion models, renowned for their capacity to model intricate distributions, present a compelling alternative.

The experimental results in Wang *et al.* conclusively demonstrate that Diffusion-QL’s state-of-the-art performance on D4RL benchmarks stems from two key factors: (1) the exceptional expressiveness of diffusion policies, and (2) the effective synergy between behavior cloning (as implicit regularization) and Q-learning guidance. Building upon these insights, our work systematically examines compu-

tational bottlenecks in Diffusion-QL while developing accelerated variants compatible with online adaptation.

2 Preliminaries and related work

Diffusion-QL Diffusion-QL[3] employs a diffusion model as its policy representation, leveraging this class of highly expressive deep generative models for decision-making. The method implements a conditional diffusion model architecture that simultaneously learns an action-value function while incorporating a value-maximization term into the diffusion model’s training objective. This formulation enables Diffusion-QL to discover optimal actions that remain proximate to the behavior policy’s distribution.

The algorithm’s exceptional performance stems from two key properties: (1) the inherent expressiveness of diffusion-model based policies, and (2) the effective integration of behavior cloning with policy improvement within the diffusion framework. Diffusion-QL demonstrates its advantages over existing approaches through a multi-modal 2D bandit case study, followed by state-of-the-art results across most D4RL benchmark tasks.

Efficient Diffusion Policy (EDP) EDP[1] is based on the denoising diffusion probabilistic model (DDPM), which learns a noise-prediction network to predict the noise used to corrupt an example.

In the forward diffusion process, a corrupted sample follows a predefined Gaussian distribution when the clean example and timestep are given. In turn, given a corrupted sample and predicted noise, EDP can approximate its clean version by leveraging the reparametrization trick. Based on this observation, to avoid the tedious sampling process, EDP use a technique called action approximation to build an action from a corrupted one, which can be easily constructed from the dataset. In this way, each training step only needs to pass through the noise-prediction network once, thus substantially reducing the training time.

EDP employs a DPM-Solver to accelerate the training, which can reduce the sampling steps from 1000 to 15 compared to diffusion-QL using DDPM sampling. As experimented, simply adding action approximation will obtain **2x** speed-up without performance loss.

The Efficient Diffusion Policy (EDP) tackles two major drawbacks of Diffusion-QL:

1. High computational costs from full Markov chain propagation during training.
2. Incompatibility with likelihood-based RL algorithms due to diffusion models’ intractable likelihood.

Diffusion Policy Policy Optimization (DPPO) DPPO[2] is an algorithmic framework for fine-tuning pre-trained diffusion-based policies in continuous control and robot learning using policy gradient methods from reinforcement learning. While PG methods were previously conjectured to be inefficient for diffusion policies in control tasks, DPPO demonstrates strong performance and efficiency.

The core idea of DPPO is to treat the denoising diffusion process as an internal Markov Decision Process (MDP), termed the “Diffusion MDP”. This Diffusion MDP is then embedded within the standard environment MDP, forming a two-layer “Diffusion Policy MDP” (MDP). The policy within this MDP has an analytically tractable Gaussian likelihood at each denoising step, making it amenable to PG updates. DPPO specifically instantiates this with Proximal Policy Optimization (PPO).

Key aspects of DPPO include:

- An advantage estimator that respects the two-level MDP structure, incorporating an environment discount γ_{ENV} and a denoising discount γ_{DENOISE} . The denoising discount downweights the contribution of noisier, earlier denoising steps.
- A state-value function $V(s_t)$ that depends only on the environment state s_t , not the full Diffusion Policy MDP state, which enhances training efficiency and stability.
- Best practices such as fine-tuning only the last few denoising steps or fine-tuning DDIM sampling, and modifications to the diffusion noise schedule for stable training and exploration.

DPPO has also shown benefits like structured exploration near the pre-training data manifold, training stability, and robust policy performance, even enabling zero-shot sim-to-real transfer.

Algorithm 1 Diffusion Q-learning

- 1: Initialize policy network π_θ (i.e., ϵ_θ network), critic networks Q_{ϕ_1} and Q_{ϕ_2} , and target networks $\pi_{\theta'}, Q_{\phi'_1}$ and $Q_{\phi'_2}$
- 2: **for** each iteration **do**
- 3: Sample transition mini-batch $\mathcal{B} = \{(s_t, a_t, r_t, s_{t+1})\} \sim \mathcal{D}$. ▷ #Q-value function learning
- 4: Sample $a_{t+1}^N \sim \mathcal{N}(0, I)$
- 5: **for** $i = N$ down to 1 **do**
- 6: Generate $\epsilon \sim \mathcal{N}(0, I)$
- 7: Compute $\mu_{\theta'}(a_{t+1}^i, s_{t+1}, i) = \frac{1}{\sqrt{1-\beta_i}} \left(a_{t+1}^i - \frac{\beta_i}{\sqrt{1-\alpha_i}} \epsilon_{\theta'}(a_{t+1}^i, s_{t+1}, i) \right)$
- 8: Compute $a_{t+1}^{i-1} \leftarrow \mu_{\theta'}(a_{t+1}^i, s_{t+1}, i) + \sqrt{\beta_i} \cdot \epsilon$
- 9: **if** $i = 1$ **then**
- 10: $a_{t+1}^{i-1} \leftarrow \mu_{\theta'}(a_{t+1}^i, s_{t+1}, i)$
- 11: **end if**
- 12: **end for**
- 13: Let a_{t+1}^0 be the final sampled action from $\pi_{\theta'}$ for s_{t+1}
- 14: Update Q_{ϕ_1} and Q_{ϕ_2} by minimizing the objective:

$$\mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}, a_{t+1}^0 \sim \pi_{\theta'}} \left[\left\| \left(r(s_t, a_t) + \gamma \min_{j=1,2} Q_{\phi'_j}(s_{t+1}, a_{t+1}^0) \right) - Q_{\phi_j}(s_t, a_t) \right\|^2 \right]$$

- 15: Sample $a_t^N \sim \mathcal{N}(0, I)$
- 16: **for** $i = N$ down to 1 **do**
- 17: Generate $\epsilon' \sim \mathcal{N}(0, I)$
- 18: Compute $\mu_\theta(a_t^i, s_t, i) = \frac{1}{\sqrt{1-\beta_i}} \left(a_t^i - \frac{\beta_i}{\sqrt{1-\alpha_i}} \epsilon_\theta(a_t^i, s_t, i) \right)$
- 19: Compute $a_t^{i-1} \leftarrow \mu_\theta(a_t^i, s_t, i) + \sqrt{\beta_i} \cdot \epsilon'$
- 20: **if** $i = 1$ **then**
- 21: $a_t^{i-1} \leftarrow \mu_\theta(a_t^i, s_t, i)$
- 22: **end if**
- 23: **end for**
- 24: Let a_t^0 be the final sampled action from π_θ for s_t
- 25: Update policy by minimizing the objective:

$$\mathcal{L}(\theta) = \mathcal{L}_d(\theta) - \frac{\alpha_{balance}}{\mathbb{E}_{(s,a) \sim \mathcal{D}}[|Q_{\phi_1}(s, a)|]} \cdot \mathbb{E}_{s \sim \mathcal{D}, a^0 \sim \pi_\theta} [Q_\phi(s, a^0)]$$

- 26: Where $\mathcal{L}_d(\theta)$ is given by:

$$\mathcal{L}_d(\theta) = \mathbb{E}_{i \sim \mathcal{U}\{1, \dots, N\}, \epsilon \sim \mathcal{N}(0, I), (s, a) \sim \mathcal{D}} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\alpha_i} a + \sqrt{1-\alpha_i} \epsilon, s, i) \right\|^2 \right]$$

- 27: $\theta' = \rho \theta' + (1 - \rho) \theta$
 - 28: $\phi'_j = \rho \phi'_j + (1 - \rho) \phi_j$ for $j = \{1, 2\}$.
 - 29: **end for**
-

Algorithm 2 EDP

Require: Input: $I, \lambda, \eta, \mathcal{D}, \epsilon_\theta(a^k, k; s), Q_\phi(s, a)$

Ensure: Output: θ, ϕ (Implicitly, the trained policy and Q-function parameters)

1: **for** $i = 0, \dots, I$ **do**

2: Sample a batch of data $\{(s, a, s')\} \sim \mathcal{D}$

3: Sample next actions for Q-network update: $a' \sim \pi_\theta(\cdot|s')$ where π_θ is parameterized by the reverse process of a conditional diffusion model:

$$\pi_\theta(a|s) = p_\theta(a^{0:K}|s) = p(a^K) \prod_{k=1}^K p_\theta(a^{k-1}|a^k, s)$$

With $p_\theta(a^{k-1}|a^k) = \mathcal{N}(a^{k-1}; \mu_\theta(a^k, k), \Sigma_\theta(a^k, k))$ where $\mu_\theta(a^k, k; s) = \frac{1}{\sqrt{\alpha^k}}(a^k - \frac{\beta^k}{\sqrt{1-\alpha^k}}\epsilon_\theta(a^k, k; s))$ and $\Sigma_\theta(a^k, k; s) = \beta^k I$.

4: Update Q_ϕ by minimizing the Temporal Difference (TD) error:

$$L_{TD}(\phi) = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[\left(r(s, a) + \gamma \max_{a'} Q_\phi(s', a') - Q_\phi(s, a) \right)^2 \right]$$

5: $\phi \leftarrow \phi - \eta \nabla_\phi L_{TD}(\phi)$

6: Approximate action for policy update (Action Approximation):

$$\hat{a}^0 = \frac{1}{\sqrt{\alpha^k}} a^k - \frac{\sqrt{1-\alpha^k}}{\sqrt{\alpha^k}} \epsilon_\theta(a^k, k; s)$$

7: Update θ by minimizing the combined loss $L_{diff}(\theta) + \lambda L_\pi(\theta)$:

$$L_{diff}(\theta) = \mathbb{E}_{k, \epsilon, (a^0, s)} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\alpha^k} a^0 + \sqrt{1-\alpha^k} \epsilon, k; s) \right\|^2 \right]$$

$$L_\pi(\theta) = -\mathbb{E}_{s \sim \mathcal{D}, \hat{a}^0} [Q_\phi(s, \hat{a}^0)]$$

8: $\theta \leftarrow \theta - \eta \nabla_\theta (L_{diff}(\theta) + \lambda L_\pi(\theta))$

9: **end for**

10: **return** Trained policy parameters θ and Q-function parameters ϕ .

Algorithm 3 DPPO

1: Pre-train diffusion policy π_θ with offline dataset $\mathcal{D}_{off}$ using BC loss:

$$\mathcal{L}_{BC}(\theta) = \mathbb{E}^{(s_t, a_t^0) \sim \mathcal{D}_{off}} \left[\left\| \epsilon_t - \epsilon_\theta(a_t^0, s_t, k) \right\|^2 \right]$$

2: Initialize value function V_ϕ

3: **for** iteration = 1, 2, ... **do**

4: Initialize rollout buffer $\mathcal{D}_{itr}$

5: $\pi_{\theta_{old}} = \pi_\theta$

6: **for** environment = 1, 2, ..., N in parallel **do**

7: Initialize state $\bar{s}_{\bar{t}(0,K)} = (s_0, a_0^K)$ in $\mathcal{M}_{DP}$.

8: **for** environment step $t = 1, \dots, T$, denoising step $k = K - 1, \dots, 0$ **do do**

9: Sample the next denoised action $\bar{a}_{\bar{t}(t,k)} = a_t^k \sim \pi_{\theta_{old}}$

10: **if** $k = 0$ **then**

11: Run a_t^0 in the environment and observe $\bar{R}_{\bar{t}(t,0)}$ and $\bar{s}_{\bar{t}(t+1,K)}$

12: **else**

13: Set $\bar{R}_{\bar{t}(t,k)} = 0$ and $\bar{s}_{\bar{t}(t,k-1)} = (s_t, a_t^k)$.

14: **end if**

15: Add $1(k, \bar{s}_{\bar{t}(t,k)}, \bar{a}_{\bar{t}(t,k)}, \bar{R}_{\bar{t}(t,k)})$ to $\mathcal{D}_{itr}$

16: **end for**

17: **end for**

18: Compute advantage estimates $A^{\pi_{\theta_{old}}}(s_{\bar{t}(t,k=0)}, a_{\bar{t}(t,k=0)})$ for $\mathcal{D}_{itr}$ using GAE:

$$\hat{A}_{\bar{t}(t,k=0)}^\lambda = \sum_{l=0}^{\infty} (\gamma\lambda)^l \bar{\delta}_{\bar{t}(t+l,k=0)}; \text{ where } \bar{\delta}_{\bar{t}(t,k)} = \bar{R}_{\bar{t}(t,k)} + \gamma_{ENV} V_\phi(\bar{s}_{\bar{t}(t+1,k)}) - V_\phi(\bar{s}_{\bar{t}(t,k)})$$

19: **for** update = 1, 2, ... num_update **do**

20: **for** batch = 1, 2, ..., B **do**

21: Sample $(k, \bar{s}_{\bar{t}(t,k)}, \bar{a}_{\bar{t}(t,k)}, \bar{R}_{\bar{t}(t,k)})$ and $A^{\pi_{\theta_{old}}}(s_{\bar{t}(t,k)}, a_{\bar{t}(t,k)})$ from $\mathcal{D}_{itr}$.

22: Compute denoising-discounted advantage

$$\hat{A}_{\bar{t}(t,k)} = \gamma_{DENOISE}^k A^{\pi_{\theta_{old}}}(s_{\bar{t}(t,0)}, a_{\bar{t}(t,0)})$$

23: Optimize π_θ using policy gradient loss:

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}_{itr}} \left[\min \left(\hat{A}_{\bar{t}(t,k)} \frac{\pi_\theta(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}{\pi_{\theta_{old}}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}, \hat{A}_{\bar{t}(t,k)} \text{clip} \left(\frac{\pi_\theta(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}{\pi_{\theta_{old}}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}, 1 - \epsilon, 1 + \epsilon \right) \right) \right]$$

24: Optimize V_ϕ using value loss:

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}_{itr}} \left[\left\| \sum_{l=0}^{T-t} \gamma_{ENV}^l \bar{R}_{\bar{t}(t+l,k)} - V_\phi(s_t) \right\|^2 \right]$$

25: **end for**

26: **end for**

27: **end for**

28: **return** converged policy π_θ .

3 Methods

Building upon the ability of EDP and DPPO to address distinct limitations of Diffusion QL, our innovation is a synergistic two-stage framework that integrates these algorithms. This approach is structured as follows:

- **Offline Pre-training with EDP:** In the initial phase, EDP is employed for robust policy and value (or Q-value) function pre-training using the available offline dataset. For the purpose

of this combined framework, EDP is utilized purely for its offline learning capabilities, without a subsequent online improvement phase by EDP itself.

- **Online Fine-tuning with DPPO:** Following the EDP pre-training, DPPO is deployed for online policy refinement through interaction with the environment. Critically, DPPO initializes its online phase by inheriting the policy network from EDP while adopting a **phased training strategy** for the value function. Our implementation proceeds through two distinct phases:

Phase (1) : Updates the state-value network ($V(s)$) only.

Phase (2) : Resumes standard DPPO training dynamics where both policy and value networks are updated concurrently.

This methodology aims to capitalize on EDP’s effectiveness in leveraging offline data to establish a strong initial policy and value estimation, which DPPO then further enhances and adapts through efficient online interaction and policy gradient updates.

4 Experiment

4.1 Reproduction of toy examples using Diffusion-QL

4.1.1 Behavior Cloning

This initial experiment is designed to evaluate the capabilities of various policy representation methods, particularly in the context of a behavior-cloning task with multimodal data.

The experimental setup, as described in the paper, defines a simple bandit task. Actions, denoted by a , are real-valued and exist in a 2-dimensional space, such that $a \in [-1, 1]^2$. An offline dataset, $\mathcal{D} = \{(a_j)\}_{j=1}^M$, is constructed using $M = 10,000$ action examples. These actions are generated by sampling from an equal mixture of four distinct Gaussian distributions. The centers (means) μ for these Gaussian distributions are specified as:

$$\mu \in \{(0.8, 0.8), (0.8, -0.8), (-0.8, -0.8), (-0.8, 0.8)\}$$

The standard deviation for each Gaussian distribution, along both dimensions, is set to $\sigma_d = (0.05, 0.05)$. This configuration results in a multimodal behavior policy.

Our primary objective in reproducing this setup is to observe and verify how effectively different policy models, with a particular focus on the diffusion-based approach, can capture and represent the four density modes of this behavior policy. This serves as a foundational step for understanding the expressiveness of diffusion policies compared to other established methods when dealing with complex, multimodal action distributions commonly found in offline datasets.

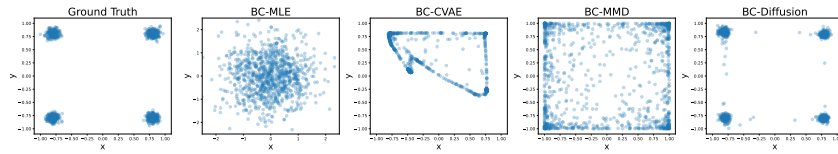


Figure 1: Equal Gaussian Mixture Distribution: BC

4.1.2 Policy Improvement

Following the behavior cloning evaluation, Section 4 of the paper extends the toy example to investigate policy improvement within an offline reinforcement learning (RL) framework. This phase utilizes the same offline dataset $\mathcal{D}$ consisting of 10,000 actions generated from the four Gaussian modes previously described. The state s for this bandit task can be considered fixed or uniform, with the learning focused on the action policy $\pi(a|s)$ and value function $Q(s, a)$.

To enable policy learning, a reward mechanism is introduced. Each action a_j in the dataset is assigned a reward r_j . According to the paper, these rewards are sampled from a Gaussian distribution. The standard deviation of this reward distribution is fixed at 0.5. The mean of the reward for any given

action is determined by its relation to a specific target "data center". As illustrated in the second row of Figure 1 in the original paper (which depicts the reward landscape), this reward structure is intentionally designed to make one of the four modes optimal. Specifically, the mode corresponding to actions around the center $(0.8, -0.8)$ is associated with higher expected rewards. This setup mimics an offline RL scenario where the true underlying reward function is unknown to the agent and must be inferred from the static dataset.

In this policy improvement phase, the performance of Diffusion Q-Learning (Diffusion-QL) is benchmarked against several prior offline RL algorithms: TD3+BC, Behavior Cloning Q-learning (BCQ), and BEAR-MMD. The paper states that all methods are trained for 1000 epochs to ensure convergence.

Our objective in reproducing this part of the experiment is to assess whether Diffusion-QL, by leveraging its expressive diffusion policy and Q-learning guidance, can effectively identify and converge to the high-reward region (the optimal mode). This contrasts with the expected performance of other methods, which, as the paper suggests, may be constrained by their policy regularization schemes or limited policy expressiveness, leading to suboptimal solutions or failure to converge to the true optimal mode. The original paper reports that Diffusion-QL successfully converges to this optimal region.

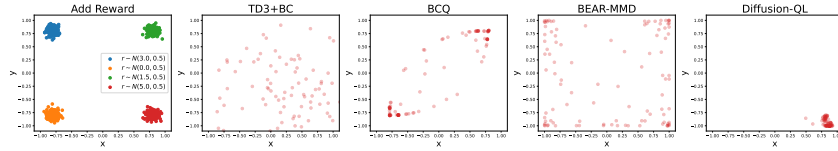


Figure 2: Equal Gaussian Mixture Distribution: QL

4.1.3 Investigating the Impact of Diffusion Steps N

A crucial hyperparameter in the Diffusion Q-Learning (Diffusion-QL) algorithm is the number of diffusion timesteps, denoted by N . This parameter determines the granularity of the diffusion process and significantly influences the model's behavior, policy expressiveness, and computational cost. Section 4 of the original paper (specifically Figure 2 and its accompanying discussion) investigates the effects of varying the value of N .

In our reproduction efforts, we also aimed to investigate the sensitivity of Diffusion-QL (or BC-Diffusion) to the parameter N . We conducted experiments where we varied N across values: $N \in \{2, 5, 10, 50\}$, utilizing the same experimental setting detailed in Sections 2.1.1 and 2.1.2. We then observed its effects on the performance of BC-Diffusion and Diffusion-QL.

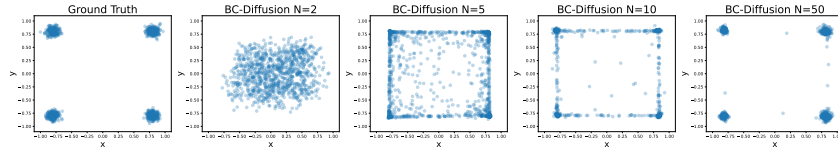


Figure 3: Modify N: BC

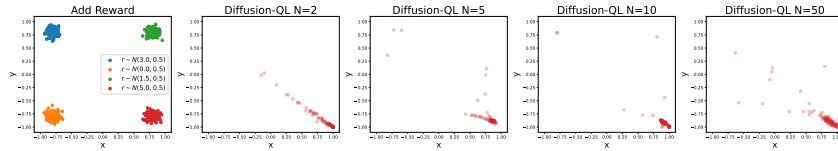


Figure 4: Modify N: QL

4.2 Extended Investigation: Other Toy Bandit Examples

Building upon our reproduction of the toy examples presented in the original paper, we further extended our investigation by designing and implementing three distinct sets of custom toy bandit examples. The primary motivation for this extension was to evaluate the robustness, adaptability, and potential limitations of Diffusion Q-Learning (and possibly other baseline methods) under scenarios that introduce increased complexity in both the underlying action distributions of the behavior policy and the associated reward landscapes.

4.2.1 Mixture Gaussian with Increased Modal Complexity

This first new toy example was designed to significantly increase the complexity of the action distribution compared to the original paper’s four-mode bandit task. The goal was to test the algorithms’ ability to handle a much larger number of modes with varying spatial arrangements and densities. The actions a remain in a 2D space, typically clipped to $a \in [-1, 1]^2$. For our experiments, we generated an offline dataset $\mathcal{D}_1$ consisting of $M_1 = 10,000$ action samples.

Action Distribution Design The action distribution in this example is a mixture of Gaussian modes sourced from three distinct geometric structures:

- **Inner Ring Modes:** A set of $N_{inner} = 16$ Gaussian modes are equally spaced on an inner circle of radius $r_{inner} = 0.5$. Each mode in this ring has a standard deviation of $\sigma_{ring} = 0.015$. Approximately 35% of the total action samples are drawn from these inner ring modes.
- **Outer Ring Modes:** A second set of $N_{outer} = 24$ Gaussian modes are equally spaced on an outer circle of radius $r_{outer} = 0.75$. These modes also have a standard deviation of $\sigma_{ring} = 0.015$ and are angularly offset relative to the inner ring modes. Approximately 45% of the total samples originate from this outer ring.
- **Corner Modes:** Finally, $N_{corner} = 4$ Gaussian modes are placed at the corners of the action space, specifically at coordinates $(\pm 0.9, \pm 0.9)$. These corner modes have a larger standard deviation of $\sigma_{corner} = 0.03$. The remaining approximately 20% of samples are drawn from these corner modes.

In total, this configuration results in $N_{total} = N_{inner} + N_{outer} + N_{corner} = 16 + 24 + 4 = 44$ distinct Gaussian modes. The samples from each mode are clipped to ensure they stay within the $[-1, 1]^2$ action space. The state s for this bandit task is considered fixed (e.g., all zeros).

Reward Function Design The reward function $R_1(a)$ for this complex action distribution was designed to be a continuous landscape formed by the sum of multiple 2D Gaussian peaks, rather than being tied to specific modes directly. This creates a scenario with multiple local optima of varying magnitudes and spreads. The primary Gaussian peaks contributing to the reward are centered at:

- Peak 1: Center $(0.8, -0.8)$, Amplitude ≈ 6.0 , Standard Deviations $(\sigma_x, \sigma_y) = (0.15, 0.15)$. This forms the dominant high-reward region.
- Peak 2: Center $(0.0, r_{outer} = 0.75)$, Amplitude ≈ 3.5 , Standard Deviations $(0.2, 0.2)$.
- Peak 3: Center $(-r_{inner} = -0.5, 0.0)$, Amplitude ≈ 2.0 , Standard Deviations $(0.15, 0.15)$.
- Peak 4: Center $(-0.7, 0.7)$, Amplitude ≈ 1.0 , Standard Deviations $(0.2, 0.2)$.

The final reward assigned to an action a is the sum of contributions from these peaks, plus a small amount of Gaussian noise with a standard deviation of $\sigma_{reward_noise} = 0.1$. This reward structure is intentionally complex and potentially misaligned with many of the action modes from the behavior policy, providing a challenging test for policy improvement algorithms to identify and exploit the true high-reward regions. The ground truth data, when visualized, often involves coloring actions by their true reward value to illustrate this landscape.

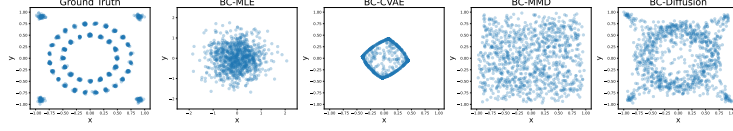


Figure 5: Complex distribution 1: Increased Modal Complexity

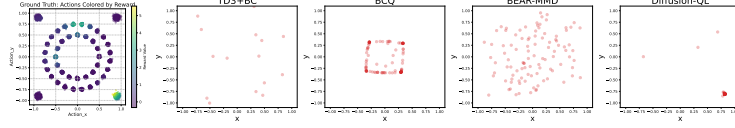


Figure 6: Complex distribution 1: Increased Modal Complexity

- Performance of BC: The policy of BC-MLE is limited to a single mode. The CVAE model even fails to exhibit mode-covering behavior, excluding the corner samples. The Tanh-Gaussian policy optimized under MMD fails to capture the true distribution. Diffusion seems to be the best among the four, but it fails to distinguish between the inner ring and outer ring.
- Performance of QL: As expected, TD3+BC, BCQ, and BEAR-MMD fail to converge to high reward regions since they cannot capture the true policy distribution. Diffusion QL, however, performs well with the guidance of Q-Learning, even though it fails to accurately capture the ground truth.

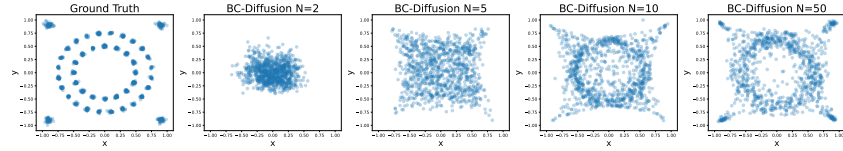


Figure 7: Complex distribution 1: Modify N BC

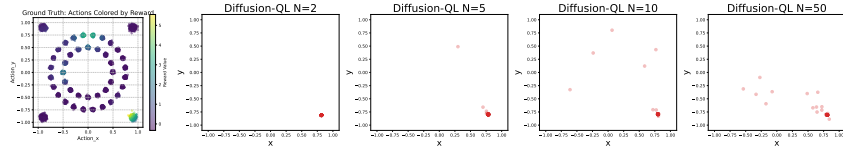


Figure 8: Complex distribution 1: Modify N QL

The influence of N:

- **N=2:** Generated points exhibit dispersed Gaussian distribution, failing to capture the four corner clusters and dual-ring structure of the true distribution.
- **N=5:** Find part of the distribution. Initial ring structures emerge, but corner clusters remain underdeveloped compared to ground truth.
- As N increases, the scattered points have decreased, and more data points are increasingly concentrated in the original distribution locations. And a complex policy distribution takes bigger N to converge.
- **N=50:** Generated samples best approximate the Ground Truth.
- The experiments validate the *positive correlation* between the number of diffusion timesteps (N) and the model's expressive capacity, where more steps enable finer-grained distribution learning through:

- Extended reverse diffusion chain for gradual distribution refinement
- Enhanced noise scheduling precision (β_t interpolation)
- Improved gradient flow through deeper temporal unfolding

4.2.2 Continuous Spiral Action Distribution with Radial Sinusoidal Rewards

This second new toy example introduces a different type of complexity, featuring a continuous, spiral-like action distribution and a reward function that depends solely on the magnitude (radius) of the action, creating concentric rings of varying reward. The actions a are in a 2D space and naturally fall within the unit circle, $a \in [-1, 1]^2$. For our experiments with this setup, we generated an offline dataset $\mathcal{D}_2$ consisting of $M_2 = 10,000$ action samples.

Action Distribution Design The action distribution is generated as follows:

- A radial component R is sampled uniformly from the interval $[0, 1)$, i.e., $R \sim U(0, 1)$.
- An angular component θ_{actual} is determined as a function of R : $\theta_{\text{actual}} = k_R \cdot R$, where the coefficient k_R is set to 6π by default.
- The 2D action $a = (x, y)$ is then derived using standard polar to Cartesian conversion:

$$x = R \cos(\theta_{\text{actual}}) = R \cos(k_R R)$$

$$y = R \sin(\theta_{\text{actual}}) = R \sin(k_R R)$$

This generation process results in a continuous action distribution forming a spiral pattern, where the angle of an action is coupled with its radius. As R increases, the action completes multiple rotations (specifically, 3 full rotations as R goes from 0 to 1 with $k_R = 6\pi$). The density of points is inherently higher towards the center ($R \approx 0$). The state s for this bandit task is considered fixed (e.g., all zeros).

Reward Function Design The reward function $R_2(a)$ in this example is designed to be dependent only on the radial distance R from the origin (where $R = \sqrt{x^2 + y^2}$ for an action $a = (x, y)$). It is defined by a sinusoidal function:

$$R_2(a) = A \sin(\omega_R R + \phi) + C$$

The default parameters for this function are:

- Amplitude $A = 0.5$
- Angular frequency on radius $\omega_R = 10\pi$
- Phase $\phi = 0.0$
- Offset $C = 0.5$

This reward structure creates multiple concentric rings of high and low rewards as R varies from 0 to 1 (specifically, 5 cycles of the sine wave since $\omega_R = 10\pi$). A key characteristic is that the reward is independent of the action's angle θ_{actual} . This setup challenges the learning algorithms to discern the radial dependency of the reward, potentially averaging over angles if the policy representation is not sufficiently expressive or if the Q-function learning is not precise. No explicit noise is added to this reward function in its generation. The ground truth data, when visualized (as in 'axs[0]' of the provided script), typically shows actions colored by their reward value, clearly illustrating these concentric reward bands overlaid on the spiral action distribution.

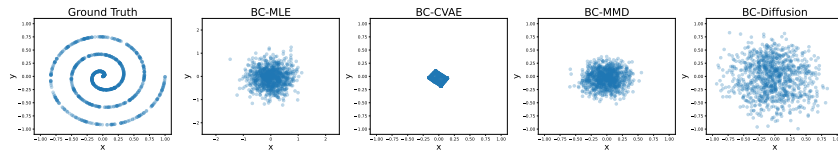


Figure 9: Complex distribution 2: Mosquito coil shape

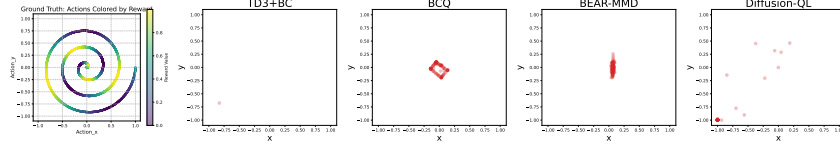


Figure 10: Complex distribution 2: Mosquito coil shape

- Performance of BC: the data points of MLE and MMD are scattered but irregular, and there is a significant difference from the true spiral distribution. The data of CAVE is more likely to be concentrated, while the diffusion method is more dispersive, which almost cover the shape of the origin distribution.
- Performance of QL: all four methods behavior poorly, pointing out their limit on complex distribution. Poor performance may also come from unsuitable hyper-parameters, which is also the direction we will investigate.

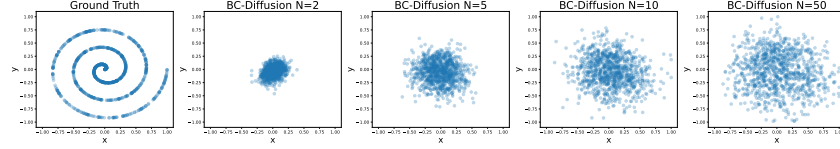


Figure 11: Complex distribution 2: Modify N BC

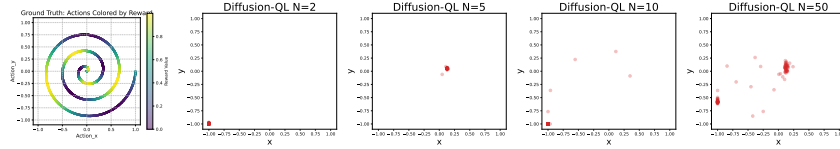


Figure 12: Complex distribution 2: Modify N QL

- For complex policy, simply increasing the size of N does not lead to a significant improvement in performance.

4.2.3 Action Distribution Forming 'SI252' with Targeted Rewards

This third custom toy example introduces a highly structured action distribution where data points form a sequence of characters, specifically "SI252". The reward function is designed to be highly targeted, with a specific small region within one of the characters yielding significantly higher rewards than the rest of the action space. This setup aims to test an algorithm's ability to explore and identify a sparse, high-reward region within a complex, multi-modal behavior policy. The actions a are in a 2D space, generally clipped to $a \in [-1, 1]^2$. The offline dataset $\mathcal{D}_3$ for this experiment consists of $M_3 = 10,000$ action samples.

Action Distribution Design The action distribution is constructed by arranging a sequence of five characters, 'S', 'I', '2', '5', '2', horizontally across the 2D action space. This process results in a complex, highly structured multi-modal distribution where the modes collectively form the visual pattern "SI252".

Reward Function Design The reward function $R_3(a)$ for this "SI252" action distribution is specifically designed to be targeted, creating a "needle in a haystack" problem:

- **High-Reward Region:** A small, specific set of modes within the central 'I' character (specifically, the two middle points of its vertical stem) are designated as high-reward modes. Samples generated from these specific modes are assigned rewards drawn from a Gaussian distribution with a mean of $\mu_{\text{high}} = 5.0$.

- **Low-Reward Regions:** All other modes that constitute the characters 'S', the remaining parts of 'I', '2', and '5' are designated as low-reward modes. Samples from these modes are assigned rewards drawn from a Gaussian distribution whose mean is randomly chosen from the set $\{1.0, 2.0, 3.0\}$.
- **Reward Standard Deviation:** For all rewards, whether from high or low-mean modes, the standard deviation for sampling the actual reward value is fixed at $\sigma_{\text{reward}} = 0.5$.

This reward structure ensures that most of the action space yields relatively low and somewhat variable rewards, while only a very precise sub-region (middle of the 'I') offers a significantly higher reward. This challenges algorithms to effectively explore the detailed structure of the behavior policy and identify the sparse high-reward signals without getting stuck in regions that are frequently visited by the behavior policy but offer suboptimal rewards. The ground truth visualization typically colors actions by their reward, highlighting this sparse reward characteristic.

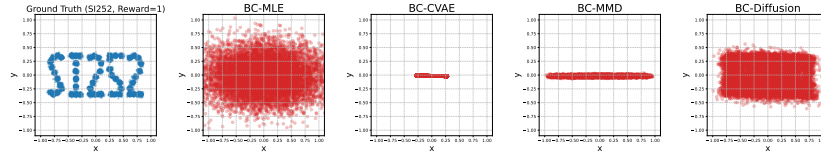


Figure 13: Complex distribution 3: clusters forming SI252

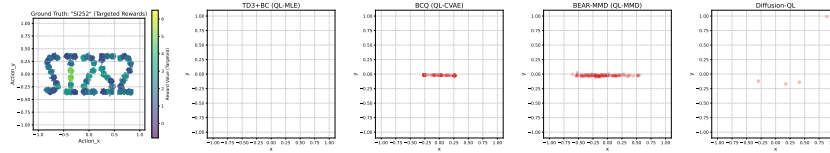


Figure 14: Complex distribution 3: clusters forming SI252

- Performance of BC: The data points are scattered and irregular.
- Performance of QL: The action distribution is almost invisible, indicating that the model fails to effectively explore or generate meaningful actions.

4.3 Gym task using Diffusion-QL

Choosing walker2d-medium-expert-v2, in which the agent controls a 2D bipedal walker with multiple joints is demanded to move straight forward without excessive energy use and falling, the algorithm stopped after 1450 epochs with the loss curves showcased below.

The declining BC and Actor loss curve with decreasing slopes indicates that the algorithm can effectively clone a successful behavior, the QL loss, finally fluctuated around $4e-5$, indicates that the Q-network could produce stable predictions about the state-action rewards, while the single peak critic loss reflects the critic network's ability to better approximate the true Q-values with self-correction, demonstrating its capability in complex non-bandit problem settings.

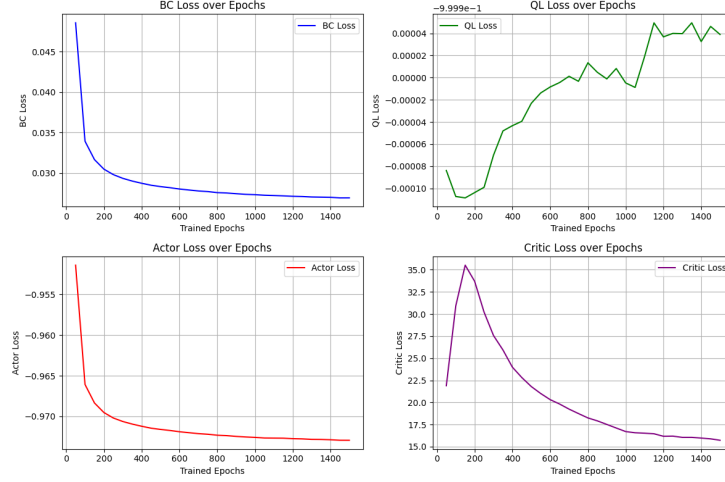


Figure 15: Diffusion QL for walker2d-medium-expert-v2

4.4 EDP improves policy

Building upon our extended investigation in Section 4.2, where we designed and analyzed various custom toy bandit examples with increased complexity, we further evaluated the performance of EDP on these same challenging distributions. As depicted in the figures below, EDP demonstrates strong performance.

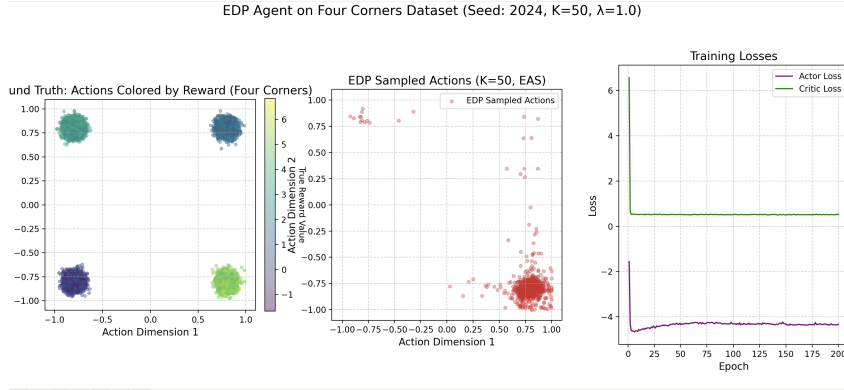


Figure 16: Equal Gaussian Mixture Distribution

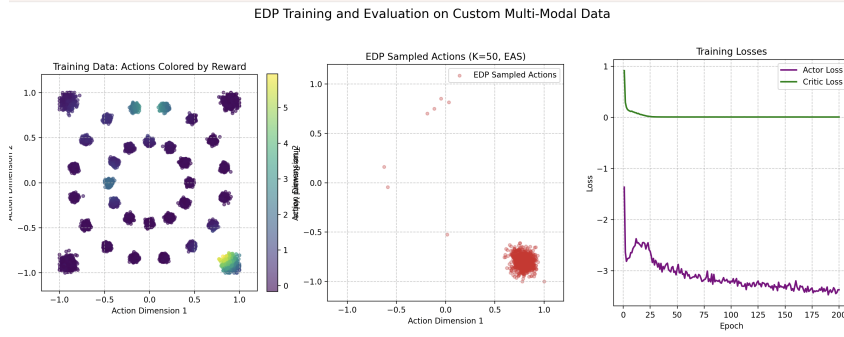


Figure 17: Increased Modal Complexity

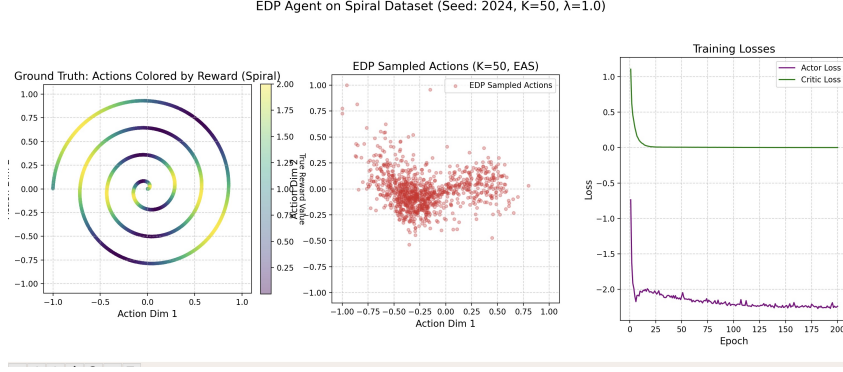


Figure 18: Mosquito coil shape

4.5 EDP-DPPO

Building upon the individual strengths of EDP and DPPO, we propose a novel two-stage framework, termed **EDP-DPPO**. This integrated approach aims to combine EDP’s efficiency in offline pre-training with DPPO’s robust online fine-tuning capabilities, seeking to achieve superior performance by leveraging the best aspects of both methodologies.

To evaluate EDP-DPPO, we conducted a comparative analysis against three established baselines: Diffusion Q-Learning (DiffQL), Efficient Diffusion Policy (EDP), and a behavior cloning-enhanced DPPO (BC-DPPO). The evaluation focuses on the cumulative reward over the first 100 episodes, as depicted in Figure 19.

Regrettably, the experimental results indicate that our proposed EDP-DPPO method does not surpass Diffusion Q-Learning (DiffQL) in terms of cumulative reward performance. While EDP-DPPO shows competitive performance compared to standalone EDP and BC-DPPO, it consistently lags behind DiffQL.

The main reasons why EDP-DPPO did not surpass DiffQL are as follows:

- DiffQL is already an extremely effective offline learning algorithm, having learned a nearly optimal policy. Consequently, there is limited room for improvement during the online interaction phase.
- Furthermore, Proximal Policy Optimization (PPO), if improperly tuned with inappropriate hyperparameters, can even lead to detrimental effects.

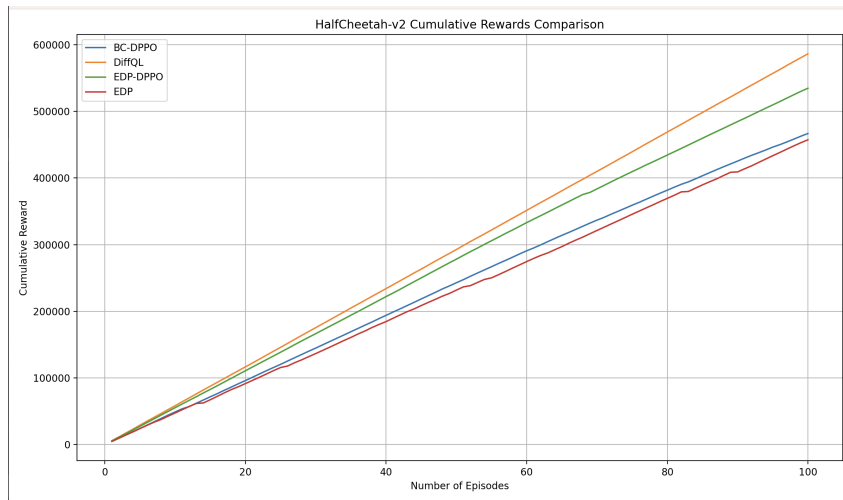


Figure 19: HalfCheetah-v2

5 Conclusion

This paper has explored and evaluated the application of diffusion models as expressive policy representations in offline reinforcement learning. We began by reviewing Diffusion Q-Learning (Diffusion-QL), which established the state-of-the-art by leveraging diffusion models for their high expressiveness and effective integration of behavior cloning with Q-learning guidance.

To address the computational inefficiencies and generality limitations of Diffusion-QL, we then detailed the Efficient Diffusion Policy (EDP). EDP significantly accelerates training by using action approximation and DPM-Solver, reducing training time from days to hours, and is compatible with various offline RL algorithms. Complementing this, Diffusion Policy Policy Optimization (DPPO) was introduced as an algorithmic framework for fine-tuning pre-trained diffusion policies using policy gradient methods, demonstrating strong performance and efficiency, especially in structured exploration and training stability.

Our experimental investigations included a comprehensive reproduction of toy examples with Diffusion-QL, demonstrating its capability in capturing multimodal action distributions and achieving policy improvement. Furthermore, we extended these investigations to custom, more complex toy bandit examples, on which EDP also demonstrated strong performance in handling intricate distributions and reward landscapes.

Finally, we introduced EDP-DPPO, a novel two-stage framework integrating EDP’s efficient offline pre-training with DPPO’s online fine-tuning. However, our comparative analysis (Figure 19) revealed that EDP-DPPO did not surpass Diffusion Q-Learning in cumulative reward performance. This outcome suggests that while DiffQL is already an extremely effective offline learning algorithm that has learned a nearly optimal policy, limiting the room for improvement during online interaction. Additionally, PPO, if improperly tuned with inappropriate hyperparameters, can even lead to detrimental effects.

6 Future Work

Future work will focus on systematically addressing the observed performance gap in EDP-DPPO. This includes:

- Investigating adaptive hyperparameter tuning mechanisms, particularly for the transition between EDP and DPPO phases, to mitigate the challenges of increased complexity.
- Further analyzing the intrinsic properties of complex offline datasets and how they influence the learning dynamics of diffusion models across different RL algorithms.

Our aim is to fully unlock the synergistic potential of combining efficient offline learning with robust online adaptation for diffusion-based policies, paving the way for more powerful and practical reinforcement learning solutions.

References

- [1] Bingyi Kang, Xiao Ma, Chao Du, Tianyu Pang, and Shuicheng Yan. Efficient diffusion policies for offline reinforcement learning. In *Advances in Neural Information Processing Systems 36*, pages 67195–67212, 2023.
- [2] Allen Z Ren, Justin Lidard, Lars L Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588v3*, 2024.
- [3] Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.